# MCMC_SEDust

*Amirhossein Rabiee , Hamidreza Mahani*

*User Manual*

This manual explains what MCMC_SEDust does, how to install and run it, what every script and function in it is doing, and how to point it at a photometric catalog of any galaxy — not only the one it was originally developed on. It is written for someone who has never used this particular code before.

Read Sections 3–4 first if you just want to get a fit running. Section 5 is the reference to come back to once you want to know what the code is actually doing at each step.

# Contents


**1 Introduction** **2**
- 1.1 What MCMC_SEDust does . . . 2
- 1.2 Who this manual is for . . . 2
- 1.3 What you need before you start . . . 2

**2 Installing MCMC_SEDust** **3**
- 2.1 Getting the code and its dependencies . . . 3
- 2.2 Getting DUSTY ready . . . 3
- 2.3 Repository layout . . . 4

**3 A first fit, step by step** **6**
- 3.1 Set up your working directory . . . 6
- 3.2 Prepare your photometry . . . 6
- 3.3 Set your priors and pick a dust template . . . 6
- 3.4 Time one model call before committing to a long run . . . 7
- 3.5 Run the fit . . . 7
- 3.6 Read the output, and decide whether to trust it . . . 7

**4 Fitting a whole catalog** **8**
- 4.1 Running it . . . 8
- 4.2 It's safe to stop and restart . . . 8
- 4.3 Parallelising over sources, not sampler walkers . . . 8
- 4.4 The summary table . . . 9
- 4.5 Comparing dust compositions . . . 9

**5 How the code works, module by module** **10**
- 5.1 dusty_kernel.py — talking to DUSTY . . . 10
- 5.2 sed_likelihood.py — data and the likelihood . . . 10
- 5.3 mcmc_dusty.py — fitting one source . . . 12
- 5.4 fit_catalog.py — fitting many sources . . . 15
- 5.5 plot_both.py — comparing compositions . . . 15

**6 Things worth knowing before you publish** **16**
- 6.1 The missing-error placeholder is an assumption, not data . . . 16
- 6.2 Tdust and tau will look correlated, and that's expected . . . 16
- 6.3 The mass-loss rate needs a luminosity correction you must apply yourself . . . 16

**7 Troubleshooting** **17**

**8 Using this for a different galaxy** **18**

**9 Command-line reference** **19**
- 9.1 mcmc_dusty.py . . . 19
- 9.2 fit_catalog.py . . . 19
- 9.3 plot_both.py . . . 19

**10 Citing this code** **20**

# 1 Introduction

## 1.1 What MCMC_SEDust does

A star losing mass late in its life often ends up wrapped in a shell of its own dust. That dust absorbs some of the star's light and re-emits it at longer wavelengths, so the handful of broadband flux measurements you have for such a star are really a mix of two things: the star's own photosphere, and the glow of the dust around it. Working out how hot the star is, how warm the dust is, and how thick the shell is, usually means comparing your photometry against a grid of pre-computed dust models and picking whichever grid point fits best.

MCMC_SEDust does this differently. Rather than searching a fixed grid, it runs the dust radiative-transfer code DUSTY on the fly, as part of a Markov-chain Monte Carlo fit, and lets three physical parameters vary continuously: the stellar temperature, the dust temperature at the inner edge of the shell, and the shell's optical depth. What you get back for each source isn't a single best-fit point but a full posterior distribution — a proper sense of how tightly each parameter is actually constrained, and how the parameters trade off against one another.

The code was originally built to fit a catalog of dusty stars in one particular galaxy, but nothing about the method is tied to that galaxy. Point it at a different photometry catalog, set priors that make sense for the population you're studying, and the same five scripts do the same job.

## 1.2 Who this manual is for

This assumes you know some Python and some basic astrophysics — what an SED is, roughly what an MCMC fit does, what "radiative transfer" means — but it does not assume you have used DUSTY, emcee, or this code before. Every script is explained, and so is every function inside those scripts that matters for actually using the pipeline.

## 1.3 What you need before you start

| You need | Notes |
|---|---|
| A compiled DUSTY V2 executable | Ivezić & Elitzur (1997); Elitzur & Ivezić (2001). MCMC_SEDust runs DUSTY as an external program — it does not include or build it for you. |
| A DUSTY input template | Fixes the dust composition, grain sizes, and shell geometry for the model you want to fit. This is not touched by the fitting code — only the three sampled parameters change from one evaluation to the next. |
| A photometric catalog | One file per source: wavelength, flux, and a flux uncertainty per band. See Section 3.2 for the exact format. |
| Python 3.6 or newer, with `numpy`, `matplotlib`, `emcee` and `corner` | `pip install -r requirements.txt` from the repository handles this. |

# 2 Installing MCMC_SEDust

## 2.1 Getting the code and its dependencies

```
git clone https://github.com/AmirhosseinRabiee/MCMC-SEDust.git
cd MCMC_SEDust
pip install -r requirements.txt
```

## 2.2 Getting DUSTY ready

DUSTY is the engine this whole pipeline is built around: every likelihood evaluation MCMC_SEDust performs is a live call out to DUSTY (Ivezić & Elitzur 1997; Elitzur & Ivezić 2001), so nothing here runs without a working DUSTY installation of your own — there's no way around compiling it first. The repository's `DUSTY_TEST/` folder has the source and every support file you need for this, arranged and ready to compile; every command below was tested end to end against exactly that folder, from a clean source download through to a successful run.

### What you need for compiling

DUSTY V2 is a single Fortran 77 source file, `dustyV2.f`. One other file has to sit next to it *only for the compile step*: `userpar.inc`, which fixes the sizes of DUSTY's internal arrays (the radial grid, the wavelength grid, and so on) and gets compiled directly into the program. The shipped defaults work fine for an ordinary spherical-shell run and essentially never need changing; if you ever do change them, you have to recompile before the change takes effect.

### Compile it

DUSTY's own manual gives the compile command as `f77 dustyV2.f -o dusty` — from an era when `f77` was the standard name for a Unix Fortran compiler. On a modern Linux system that compiler is `gfortran`, and running that command as-is will fail outright on any reasonably current gfortran (version 10 or newer): it does stricter argument checking by default than existed when this code was written, and it treats one particular subroutine call as a hard error rather than letting it through. The fix is one flag, telling the compiler to accept the source as legacy Fortran instead of holding it to a modern standard:

```
gfortran -std=legacy dustyV2.f -o dusty
```

Run this from the folder containing both `dustyV2.f` and `userpar.inc`. You'll see a handful of harmless warnings scroll by — things like `PAUSE statement` and `Arithmetic IF` being flagged as deleted legacy features — none of which stop the build. A few seconds later you'll have an executable called `dusty` in that same folder.

The first time you run it, you may also see:

```
Note: The following floating-point exceptions are signalling:
IEEE_UNDERFLOW_FLAG IEEE_DENORMAL
```

That's informational, not an error. It's glibc reporting a floating-point underflow that happens naturally somewhere in the radiative-transfer calculation; it doesn't affect the output.

### What else has to be in the run directory

The executable on its own isn't enough. At *run* time — separately from the compile step above — DUSTY also needs a set of support files sitting next to it: the wavelength grid (`lambda_grid.dat`), a few reference spectra and opacity tables (`kurucz10.dat`, `ism-stnd.dat`, `taugrid.dat`, `collapse.dat`, `quasar.dat`). All of these come from the same source distribution as `dustyV2.f`. This whole collection — the executable plus every one of these files plus your template — is the "DUSTY working directory" referred to throughout this manual (Section 2.3 and Section 3.1); `dustyV2.f` and `userpar.inc` themselves don't need to be there, they were only needed for the one-off compile.

The repository's `DUSTY_TEST/` folder has all of this pre-arranged, plus a small generic example (see below) to confirm your compile works — copy its contents into your working directory as a starting point, or point `-dusty-dir` at it directly.

One file you do *not* need to manage by hand: DUSTY's manual describes a master file, `dusty.inp`, that tells it which input file to process. `DustyKernel` (Section 5) writes this file itself, fresh, before every single call — you'll see it appear in your working directory, but nothing in this pipeline ever asks you to create or edit it yourself.

### Checking it actually works

Before trusting a fresh compile to a real fit, run it once on the small generic example, `verify_compile.inp`, inside `DUSTY_TEST/` — a self-contained model with plain round-number parameters, not a research template, whose only job is to confirm the compile works:

```
cd DUSTY_TEST
gfortran -std=legacy dustyV2.f -o dusty
echo "verify_compile" > dusty.inp
./dusty
```

A successful run produces `verify_compile.out` and `verify_compile.stb` in the same folder. If both appear, your DUSTY build is ready, and you can point MCMC_SEDust at this same folder (or copy its contents alongside your own template) to start fitting.

**Why compile from source rather than copy a working binary**

A compiled DUSTY executable is a dynamically linked binary tied to the particular C library it was built against. A copy that runs fine on one machine can simply refuse to start on another with a different Linux distribution or glibc version — a laptop versus a shared server is a common case where this bites. Recompiling from source with the one command above takes a few seconds and sidesteps the problem entirely, so it's worth doing fresh on every machine rather than copying a binary between them.

## 2.3 Repository layout

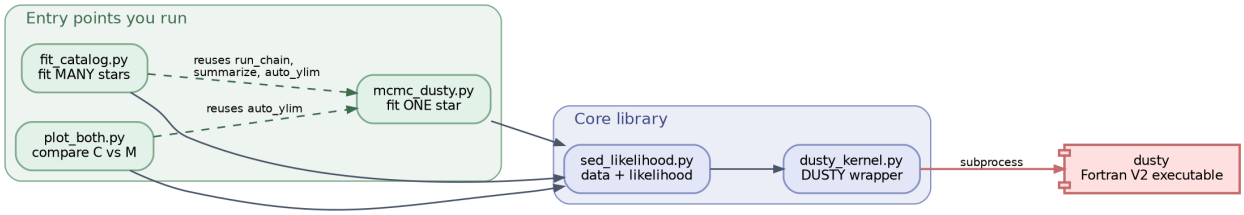



Figure 1: How the five modules relate. Dashed arrows mean "imports and reuses."

| File | What it does |
|---|---|
| `mcmc_sedust/dusty_kernel.py` | Wraps the DUSTY executable so Python can call it like a function. |
| `mcmc_sedust/sed_likelihood.py` | Loads photometry and turns a model spectrum plus the data into a number the sampler can maximise. |
| `mcmc_sedust/mcmc_dusty.py` | Fits one source. Also the shared library the other two scripts import from. |
| `mcmc_sedust/fit_catalog.py` | Fits many sources: resumable, parallel, one combined summary table. |
| `mcmc_sedust/plot_both.py` | Overlays two dust-composition fits for one source and says which fits better. |
| `DUSTY_TEST/` | DUSTY's own source, compiled-in support data, and a generic verification example — see Section 2.2. |

**These five files must stay together**

`fit_catalog.py` and `plot_both.py` both import `mcmc_dusty.py` directly, so a mismatched set of files is the single most common source of confusing errors. Update all five together. Wherever you run `python3` from is where the output files land — not the folder the scripts themselves live in — so you can invoke them from your DUSTY working directory by full or relative path, exactly as shown throughout this manual.

# 3 A first fit, step by step

The best way to get a feel for the pipeline is to fit one source by hand before turning it loose on a whole catalog. This section goes through that in the order you would actually do it.

## 3.1 Set up your working directory

Before running anything, gather three things in one folder: the compiled DUSTY executable, its support files, and the template `.inp` file that fixes the dust model (Section 2.2). If you want to compare two dust compositions — carbon-rich and oxygen-rich, say — keep two templates, one for each, and fit each in turn. Alongside these, keep your photometric catalog, laid out as one file per source.

## 3.2 Prepare your photometry

Each source needs its own file, one row per photometric band. Two input layouts are supported, and you tell the code which one you're using with `--format`.

| Format | Columns | When to use it |
|---|---|---|
| `fnu_jy` (recommended) | wavelength [μm], flux density $F_\nu$ [Jy], fractional error on wavelength, fractional error on $F_\nu$ | Most catalogs report flux density in Jansky. The loader converts to $\lambda F_\lambda$ for you. |
| `lamflam` | wavelength [μm], $\lambda F_\lambda$ directly | Use this if your catalog already gives you $\lambda F_\lambda$ rather than $F_\nu$. |

A minimal `fnu_jy` file, comma-separated and with no header row, looks like this:

```
0.55,2.10e-06,0.05,0.10
0.77,6.80e-06,0.05,0.09
1.24,1.95e-05,0.04,0.08
1.65,2.60e-05,0.04,0.08
```

**If a band has no listed uncertainty**

Real catalogs often have a flux measurement in some band with no published error — commonly the faintest or most blended bands, which are frequently the ones that matter most for constraining the dust temperature. Rather than dropping these points, the loader assigns them a declared, fixed fractional error (15% by default, set in `MISSING_FLUX_ERR` at the top of `sed_likelihood.py`) so they still inform the fit without an unverified number dominating it. Every time this happens it is printed to the screen and written into the output files, so the substitution is never invisible. Change the one constant, or better, add the real uncertainties to your catalog, and it stops happening on its own.

## 3.3 Set your priors and pick a dust template

Two things need deciding before you fit anything, and both are judgement calls about the population you're studying, not something the code can work out for you. First, the prior ranges: the minimum and maximum you're willing to let the sampler explore for the stellar temperature, the dust temperature, and the optical depth. Set them wide enough to contain the true answer but not so wide that the sampler wastes most of its time in territory you already know is implausible. Second, the dust template itself: the grain composition, size distribution, and shell geometry that DUSTY will hold fixed while it varies the three sampled parameters. These are properties of the dust model you've chosen to test, not something the fit adjusts.

### 3.4 Time one model call before committing to a long run

A single DUSTY evaluation typically takes several seconds to tens of seconds, and a full fit calls it thousands of times, so that one number — the time for a single call — tells you almost everything about how long a fit, and later a whole catalog, will take. Running `mcmc_dusty.py` without `--mode full` does exactly this: it times one DUSTY call and prints a short table estimating the cost of a few different chain lengths, without asking you anything else. It's worth doing this once before launching your first real fit, and again if you change your dust template, since the per-call time depends on the model settings.

### 3.5 Run the fit

A real fit uses `--mode full`. Without any further flags it will ask you, one at a time, for the prior ranges and the sampler settings; pass any of them on the command line and it skips asking for that one.

```
python3 mcmc_sedust/mcmc_dusty.py --comp M --mode full \
    --data my_star.csv --star star1 --format fnu_jy \
    --tstar 2000 4000 --tdust 100 1400 --tau 0.01 2 \
    --nwalkers 16 --nburn 100 --nprod 300
```

Internally, this loads the photometry, builds the model wrapper and the likelihood, scatters a set of walkers inside the prior box, runs them through a burn-in phase that gets discarded, then a production phase that becomes the answer. Section 5 explains what each of these steps is actually doing; for now, what matters is what comes out the other end.

### 3.6 Read the output, and decide whether to trust it

Four files appear for this source: a chain file holding every posterior sample, a short text summary with the median and 16th–84th-percentile range for each parameter, a corner plot showing all three parameters against each other, and a plot of the best-fitting model laid over your data points.

Three numbers, printed at the end of every run, tell you whether that fit is actually worth using — and it's worth checking them for every source, not only the ones that look strange. The **acceptance fraction** should sit somewhere around a fifth to a half; a value near zero means the walkers got stuck and never really moved, and everything from that chain should be discarded. The **autocorrelation time** tells you how many steps it takes the chain to forget where it started; you want your total chain length to be several tens of times longer than this — fifty times is a reasonable target — or the posterior hasn't properly settled down yet. And the **corner plot** is worth a look on its own terms: two separate clumps instead of one, or a parameter pinned right against the edge of its prior box, usually means the walkers split into two groups that never found each other, which a longer burn-in often fixes.

It's also worth expecting some correlation between the dust temperature and the optical depth rather than being puzzled by it: a cooler, thicker shell can produce almost the same spectrum as a warmer, thinner one, so the two trade off against each other. That shows up as a diagonal smear in the corner plot and wider individual error bars than you might expect — it's telling you something genuine about what your photometry can and can't pin down, not indicating a mistake.

**A cheap way to check your sampler settings**

If you're not sure whether a fast, inexpensive set of walkers/burn-in/ production steps is good enough, fit the same source again with a noticeably longer chain and compare. If the two runs agree within their own uncertainties, the cheaper settings are safe to use across the rest of the catalog; if they don't agree, use the longer settings. This is a more reliable guide than any rule of thumb, and it only costs one extra fit.

# 4 Fitting a whole catalog

Once a handful of sources look right, the same sequence — load, wrap, build the likelihood, sample, summarise — gets repeated for every source in your catalog, and for every dust composition you want to test. Doing that by hand, one launch per source, is not realistic once you have more than a few dozen; `fit_catalog.py` exists to do it for you.

## 4.1 Running it

```
python3 mcmc_sedust/fit_catalog.py --comp both \
    --data-dir my_catalog --first 1 --last 200 \
    --out-dir results --nwalkers 16 --nburn 100 --nprod 300 \
    --parallel-stars 64
```

This expects your photometry files named by number (`my_catalog/1.csv`, `my_catalog/2.csv`, and so on) and fits every one from `--first` to `--last`. `--comp both` fits each source in both dust compositions and pools all of that work into a single queue, which matters once parallelism is involved (Section 4.3). Everything lands in `--out-dir`.

## 4.2 It's safe to stop and restart

Before starting a fit, the script checks whether that source already has a finished result sitting in the output folder, and skips it if so. So if a job gets interrupted — a server reboot, a time limit, anything — launching the exact same command again just continues from where it left off rather than redoing everything. Use `--redo` if you deliberately want to overwrite existing results, and `--only 5,37,140` to re-fit a specific list of sources.

## 4.3 Parallelising over sources, not sampler walkers

This is the one detail that makes the difference between a catalog finishing in days rather than months, so it's worth understanding properly. Inside a single fit, the sampler can only usefully employ about half as many processor cores as it has walkers — that's a property of how the sampling algorithm proposes moves, not something tunable. But different sources are completely independent of one another, so instead of trying to squeeze more speed out of one fit with more cores, `--parallel-stars` hands whole fits to different cores at once: one source's complete chain on one core, the next source's complete chain on the next core, and so on.

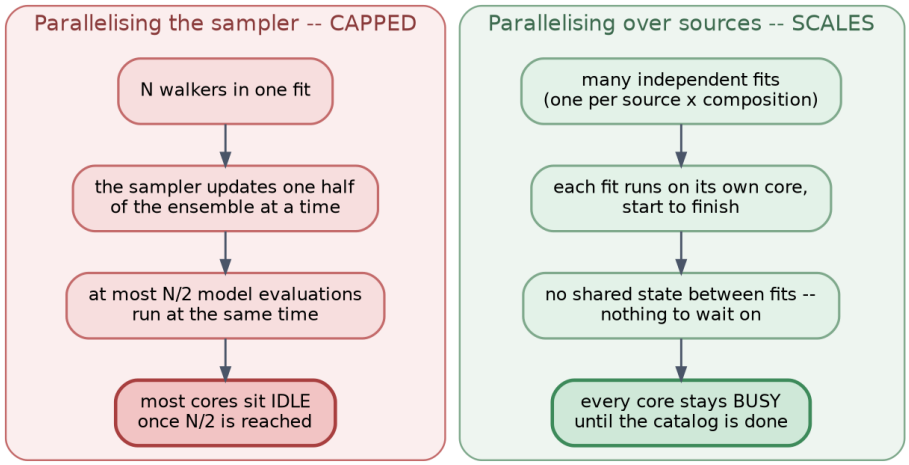



Figure 2: Why parallelising over sources scales and parallelising the sampler alone does not.

On a machine with many cores this is the difference between a catalog finishing in a couple of

days rather than several weeks, and it changes nothing about any individual result — each source is fit exactly as it would be fit on its own, just at the same time as everything else. Each (source, composition) pair gets its own random seed, so the parallel chains are properly decorrelated from one another.

## 4.4 The summary table

Alongside the usual per-source files, one combined `catalog_summary.csv` accumulates one row per source per composition: median parameters and their credible intervals, fit quality, any derived quantity the model reports, and a status flag.

| Column | Meaning |
|---|---|
| `star, model` | source identifier and composition (C or M) |
| `Tstar_med, Tstar_lo, Tstar_hi` | median and 16th/84th percentiles |
| `Tdust_med/lo/hi, tau_med/lo/hi` | same, for the other two parameters |
| `shift, chi2` | the fitted normalisation and goodness of fit |
| `Mdot` | mass-loss rate as reported by DUSTY (see Section 6.3) |
| `acceptance` | mean acceptance fraction — check before trusting a row |
| `n_assumed_err` | how many points in this fit used the placeholder error |
| `status` | 'ok', or why the fit failed |

Check the status column before anything else. A handful of sources in any real catalog will fail for one reason or another — too few usable photometric bands, or an optical depth so extreme the radiative-transfer solution won't converge — and it's far better to have those flagged clearly in one column than to notice, much later, that a plot is quietly full of nonsense.

```
grep -v ",ok," results/catalog_summary.csv
```

## 4.5 Comparing dust compositions

If you've fitted a source in both compositions, `plot_both.py` loads both finished chains, draws both best-fit curves over the photometry, and reports which one gives the lower $\chi^2$ — a quick way to see whether a given source's dust looks more carbon-rich or oxygen-rich before doing anything more quantitative with the whole catalog.

```
cd results
python3 ../mcmc_sedust/plot_both.py --star 1 \
    --data ../my_catalog/1.csv --format fnu_jy
```

# 5 How the code works, module by module

This section is the one to come back to once you want to know what the code is actually doing, not just which command to type. It follows the five files in the order data flows through them.

## 5.1 dusty_kernel.py — talking to DUSTY

Everything that actually calls the radiative-transfer code lives here, behind one small class, `DustyKernel`. From the outside it looks like an ordinary function: give it three numbers and a composition, and it hands back a spectrum.

```
lam, sed = kernel.run(Tstar=2900, Tdust=100, tau=0.565,
                      composition='M')
```

Underneath, `run()` writes those three numbers into a fresh copy of your template, launches the DUSTY executable as a subprocess, and reads back the resulting spectrum file. Everything else about the model — composition, geometry, numerical accuracy — stays exactly as set in your template.

Two things about this class matter once you start running many fits at once. First, each running copy gets its own private working directory, created the first time it's needed and reused after that, so that two fits happening at the same moment never try to read or write the same file. Second, if DUSTY fails for a given set of numbers — which it can, at extreme optical depths or unusual parameter combinations, or simply if it runs past its time limit — the class reports that cleanly instead of crashing, and the likelihood (Section 5.2) treats it as "this point is not allowed" rather than stopping the whole run.

**Why each worker gets its own folder**

DUSTY reads a fixed input filename and writes fixed output filenames in whatever directory it runs in. If two processes shared that directory, they would silently overwrite each other's files, and the results would be wrong without any error being raised. A small amount of bookkeeping inside `DustyKernel` — specifically, what happens when the object is copied to a separate worker process for parallel fitting — guarantees that never happens.

## 5.2 sed_likelihood.py — data and the likelihood

This module has two jobs: turn a photometry file into a clean set of arrays, and turn a proposed model plus those arrays into a single number the sampler can climb.

### Loading the data

Two loader functions handle the two input formats from Section 3.2, and both hand back the same kind of bundle, so nothing downstream needs to know or care which one was used. Measurements are compared to the model in $\log_{10}(\lambda F_\lambda)$ space, so a fractional flux error is converted into a log-space (dex) uncertainty by dividing it by $\ln(10)$ — that converted number is what actually enters the fit.

### The normalisation is solved, not sampled

DUSTY's output is normalised in its own internal flux unit, so matching it to your observed flux level needs a vertical shift in log space, which physically stands in for the source's distance and luminosity. That shift could have been a fourth free parameter for the sampler to explore. Instead, because the best-fitting shift for any given model shape has a closed-form solution — the inverse-variance-weighted mean of the residual between the observed and model log-fluxes — it's computed directly at every step:

$$\text{shift} = \frac{\sum_i \left[ (\log F_i - \log M_i) / \sigma_i^2 \right]}{\sum_i \left[ 1/\sigma_i^2 \right]}$$

That removes an entire dimension from the problem at no extra cost — no additional DUSTY calls — and it noticeably speeds up convergence, since this shift is exactly the quantity most degenerate with distance.

**Forming chi-squared**

With the shift in hand, the model is interpolated onto the observed wavelengths in log-log space, and the fit quality follows directly:

$$\chi^2 = \sum_i \left[ \frac{\log F_i - \log M_i - \text{shift}}{\sigma_i} \right]^2$$

The object that ties all of this together, `SEDLikelihood`, also enforces the prior box: a proposal outside the ranges you set is rejected immediately, without ever calling DUSTY, and a proposal for which DUSTY fails is treated the same way. Either case becomes a log-probability of $-\infty$, which the sampler simply steps away from.

### 5.3 mcmc_dusty.py — fitting one source

This is the script you run directly for a single fit, and it is also the shared library the other two scripts import from, so its behaviour is consistent everywhere.

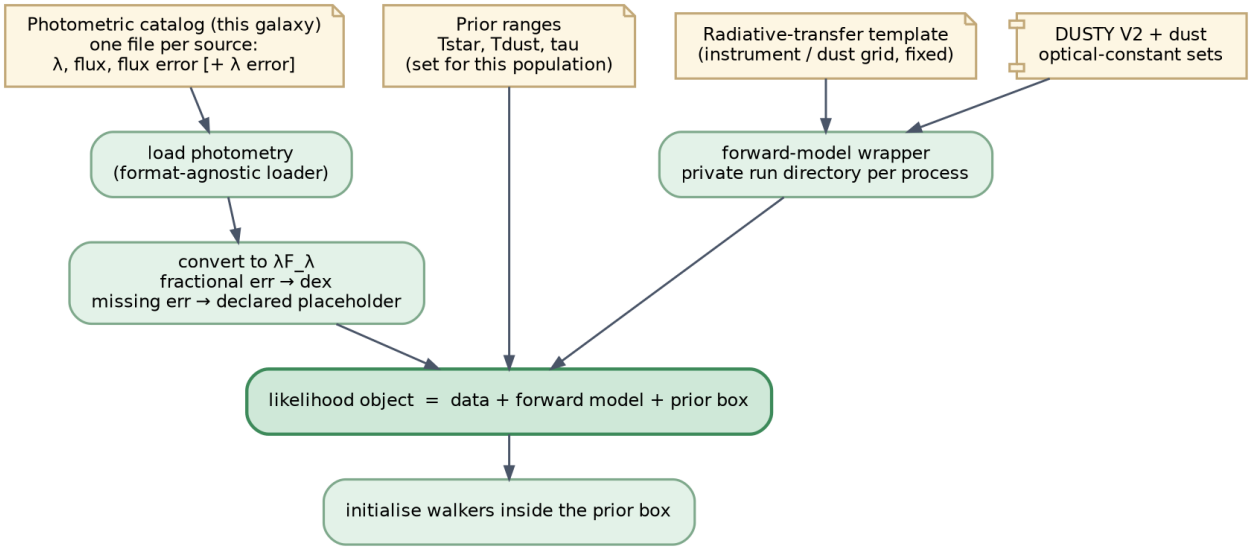



Figure 3: Setup: everything that happens once, before sampling starts.

The prior ranges and template are combined with the photometry into one `SEDLikelihood` object, and a set of walkers is scattered randomly inside the prior box to begin from.

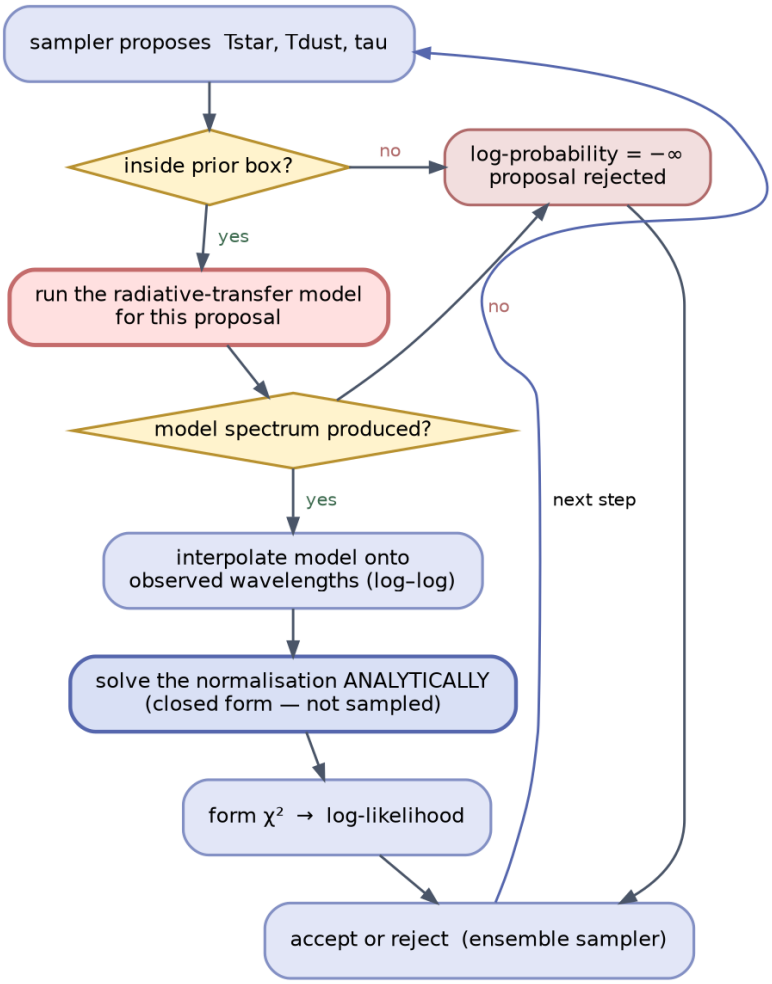



Figure 4: One step of the sampler. A rejected or failed proposal never stops the chain.

This loop repeats for the burn-in phase, then the production phase. emcee's ensemble sampler moves the walkers by comparing half the ensemble against the other half, which is also the reason its parallelism has a ceiling — covered properly in Section 4.3.

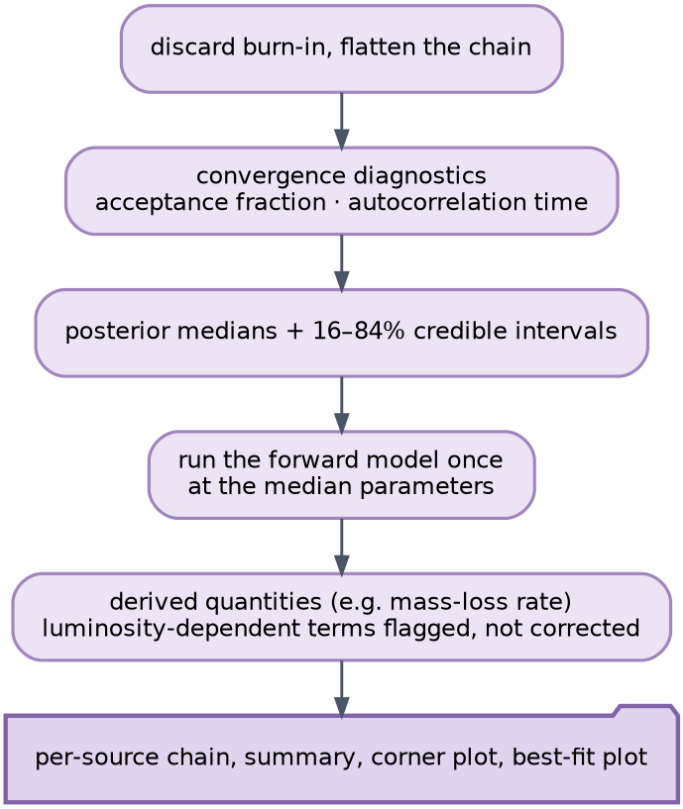



Figure 5: Once the chain finishes: everything you actually keep comes from one more DUSTY run, at the median parameters.

Running the model one final time at the posterior median is deliberate: it means the curve you see plotted and any derived number you quote (a mass-loss rate, for instance) always come from the exact same evaluation, rather than being pieced together from different samples.

**The default mode is 'quick', not 'full'**

If `mcmc_dusty.py` runs without asking you about walkers or steps, this is why: `--mode` defaults to a short, fixed-length timing run (Section 2.4) rather than a real fit. Pass `--mode full` whenever you actually want one.

### 5.4 fit_catalog.py — fitting many sources

Two passes. The first builds the list of work to do, skipping any source that already has a finished result and recording any file that's missing. The second pass runs what's left, either one at a time or through a pool of parallel workers when `--parallel-stars` is greater than one, as described in Section 4.3.

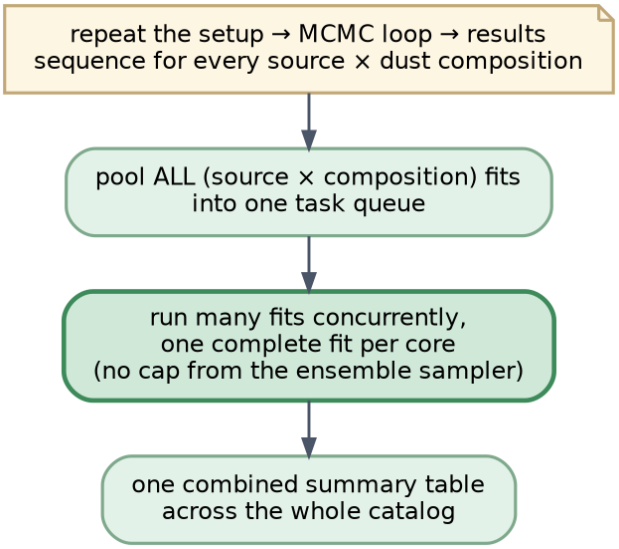



Figure 6: The setup/sampling/results sequence from the previous subsection repeats once per source and per composition; this is the only place parallel workers are involved.

Only the main process ever writes to `catalog_summary.csv`, so parallel workers can never collide while writing results, and before launching, the script times one DUSTY call itself and prints an estimate of the whole catalog's total run time — including a predicted finish time — so you know what you're committing to before you start.

### 5.5 plot_both.py — comparing compositions

Loads the two saved chains for a source, takes the posterior median from each, runs the model once more for each composition, and plots both curves against the data with a $\chi^2$ comparison. If only one composition has been fitted so far, it plots that one alone and says so.

# 6 Things worth knowing before you publish

## 6.1 The missing-error placeholder is an assumption, not data

**Recap from Section 3.2**

Photometry points with no listed error are fitted using a declared placeholder fractional error (15% by default). It is printed on screen, written into every affected summary file, and counted in the `n_assumed_err` column of the catalog table — so before reporting results, it's worth checking that column and being able to say, honestly, how many of your points relied on it.

## 6.2 Tdust and tau will look correlated, and that's expected

A cooler shell at higher optical depth can reproduce almost the same observed spectrum as a warmer, thinner one, so the two parameters genuinely trade off against each other given typical broadband photometry. Expect a tilted, banana-shaped region in that panel of the corner plot, and correspondingly wide individual error bars on both parameters — that's the data telling you the truth about what it can and can't distinguish, not a sign that something has gone wrong.

## 6.3 The mass-loss rate needs a luminosity correction you must apply yourself

DUSTY reports a mass-loss rate in its output, computed under the assumption of a fixed reference luminosity (commonly $10^4\,L_\odot$). The pipeline records that number exactly as DUSTY reports it, without attempting to correct it. Converting it into a true physical mass-loss rate for a given source means scaling by the square root of the ratio of that source's real luminosity to the reference value — which needs the source's actual luminosity, and therefore a distance to your galaxy. That correction is best applied once, after fitting, using whatever distance you've adopted, rather than being silently baked into the pipeline.

# 7 Troubleshooting

| Symptom | Likely cause / what to do |
|---|---|
| File not found for your data | Check the path you gave `--data` or `--data-dir` is relative to where you launched python3, not to where the scripts live. |
| Not asked about walkers or steps | Add `--mode full`. Without it, `mcmc_dusty.py` runs the short timing-only 'quick' mode (Section 2.4). |
| Acceptance fraction is 0.000, shift/chi2 are nan | Almost always a non-finite value reaching the likelihood — check your CSV for stray blanks or unhandled nan values outside the flux-error column (Section 3.2 covers the error column specifically). |
| AttributeError on an unexpected missing attribute | The five files are out of sync with each other — update all five together, from the same source. |
| Nothing prints for a long time in a parallel catalog run | Expected: parallel workers run quietly. Look for the periodic heartbeat line showing elapsed time and sources completed. |
| A catalog run is the same speed on many cores as on a few | Too few walkers relative to your core count if you are NOT using `--parallel-stars` (Section 4.3), or you forgot to set `--parallel-stars` at all. |
| The best-fit curve is missing from a plot | DUSTY failed at the median parameters for that fit; check the `status` column for that row in the summary table. |
| Only one composition appears in `plot_both.py` | The other composition's chain file does not exist yet for that source — fit it first. |
| emcee complains about the number of walkers | It needs at least twice as many walkers as free parameters (6, here) — use more. |

# 8 Using this for a different galaxy

Everything in Sections 3–5 is independent of which galaxy you are studying. What changes from one target to the next is entirely contained in the table below — nothing about the fitting method, the likelihood, the sampler, or the parallel batch processing needs to change.

| Must be supplied per target | Notes |
|---|---|
| Photometric catalog | One file per source, in the format from Section 3.2. |
| Prior ranges for Tstar, Tdust, tau | Should bracket the physically plausible range for the population and dust environment you are studying. |
| Dust template(s) | Grain composition, geometry and accuracy settings for the composition(s) you want to fit; match an existing study's settings if you intend to compare results with it. |
| Placeholder error fraction | The single value in `MISSING_FLUX_ERR` (Section 3.2); revise or remove once your catalog's error coverage is complete. |
| Distance / luminosity | Not needed to obtain the fitted shape parameters, but needed to convert the fitted normalisation into a physical luminosity, or to apply the correction described in Section 6.3. |

# 9 Command-line reference

## 9.1 mcmc_dusty.py

| Flag | Meaning |
|---|---|
| `--comp C\|M` | dust composition to fit |
| `--mode quick\|full` | quick = timing only (default); full = a real fit |
| `--data, --star` | the photometry file and a name for the output files |
| `--format fnu_jy\|lamflam` | input file layout (Section 3.2) |
| `--tstar/--tdust/--tau LO HI` | prior box for the three parameters |
| `--nwalkers/--nburn/--nprod` | sampler size |
| `--nproc N` | cores for walker-level parallelism (Section 4.3) |
| `--no-prompt` | never ask questions; use flags/defaults only |

## 9.2 fit_catalog.py

| Flag | Meaning |
|---|---|
| `--comp C\|M\|both` | both pools every fit into one queue |
| `--data-dir, --first, --last` | which numbered files to fit |
| `--parallel-stars N` | run N complete fits at once, one per core |
| `--out-dir DIR` | where every output file goes |
| `--only 5,37,140` | fit just these source numbers |
| `--redo` | re-fit even if a result already exists |
| `--no-prompt` | required for unattended/background runs |

## 9.3 plot_both.py

| Flag | Meaning |
|---|---|
| `--star, --data, --format` | same meaning as in `mcmc_dusty.py` |
| `--c-chain, --m-chain` | override the expected chain filenames |

## 10 Citing this code

See `CITATION.cff` in the repository for how to cite MCMC_SEDust itself. Please also cite the software it depends on and wraps:

- DUSTY — Ivezić, Z. & Elitzur, M. 1997, MNRAS, 287, 799; Elitzur, M. & Ivezić, Z. 2001, MNRAS, 327, 403
- emcee — Foreman-Mackey, D., Hogg, D. W., Lang, D., & Goodman, J. 2013, PASP, 125, 306, building on the affine-invariant ensemble sampler of Goodman, J. & Weare, J. 2010, CAMCoS, 5, 65
- corner — Foreman-Mackey, D. 2016, JOSS, 1, 24